

Lab 25 - Expected verification output

Run from the lab directory:

```
bash verification/checks.sh
```

Honesty note

checks.sh is read-only. It creates nothing, patches nothing and deletes nothing, and it touches kube-system only with get. The one command that looks like a write - `kubectl apply --dry-run=server -f broken/` - is a server-side dry run: the API server validates the objects through the full admission chain and then discards them. Nothing is persisted.

The most important check in this file

```
PASS kubeconform: all broken/ manifests pass strict schema validation
PASS server-side dry-run: the API server ACCEPTS all broken/ manifests
```

These two lines assert that the deliberately-broken manifests are valid. That is not a contradiction - it is the entire pedagogical contract of this lab. The breakage must be semantic, never structural:

File	Ticket	What is broken	Why it is still schema-valid
01-depot-api-imagepull.yaml	DEP-1041	Image tag misspelt 1.27-alpne	image is a free-form string. No schema can know which tags exist.
02-shipment-worker-crashloop.yaml	DEP-1042	Command runs a binary absent from the image	command is a string array. No schema can inspect image contents.
03-manifest-indexer-pending.yaml	DEP-1043	Requests 64 CPU and 512Gi memory	Both are well-formed resource.Quantity values. The schema has no idea how big your nodes are.
04-depot-tracker-no-endpoints.yaml	DEP-1044	Service selects app=depot-tracking; Pods are app=depot-tracker	A selector is a valid map of strings. Kubernetes never requires a selector to match anything.

If either of those two checks ever fails, the lab is broken: the learner would see `kubectl apply` rejected at admission and would never observe the runtime symptom the ticket describes.

Fully passing run - after Part 7

```

== Namespace isolation
PASS namespace kcna-lab25 exists
PASS Pod Security Admission enforce=baseline

== broken/ manifests are schema-valid (semantic breakage only)
PASS found 4 broken manifests
PASS kubeconform: all broken/ manifests pass strict schema validation
PASS server-side dry-run: the API server ACCEPTS all broken/ manifests

== Part 1 - known-good baseline
PASS deployment/depot-portal (reference workload) exists
PASS reference workload has 2 ready replica(s)

== Part 7 - all four tickets repaired
PASS every Pod in kcna-lab25 is Running
PASS depot-api: 1/1 ready
PASS shipment-worker: 1/1 ready
PASS manifest-indexer: 1/1 ready
PASS depot-tracker: 2/2 ready
PASS DEP-1041 fixed: image is nginx:1.27-alpine
PASS DEP-1042 fixed: shipment-worker has 0 restarts
PASS DEP-1043 fixed: manifest-indexer scheduled onto kind-control-plane
PASS DEP-1044 fixed: Service selector is app=depot-tracker
PASS DEP-1044 verified: EndpointSlice carries 2 backend address(es)

== Worksheets
PASS data/incident-queue.csv present
PASS data/triage-worksheet.csv present
PASS triage worksheet header intact
PASS worksheet has a row for DEP-1041
PASS worksheet has a row for DEP-1042
PASS worksheet has a row for DEP-1043
PASS worksheet has a row for DEP-1044

== Blast-radius guard - kube-system untouched
PASS no lab-25 objects in kube-system (Part 6 was read-only)
PASS control plane still reports readyz=ok

SUMMARY: 26 passed, 0 failed, 0 skipped

```

Exit code 0.

Expected run DURING Part 2-6 - before the fixes

This is what a correct mid-lab run looks like. These failures are the lab working as designed, not defects:

```

== Part 7 - all four tickets repaired
  FAIL  Pods not Running: depot-api-7d4b9c6f85-x2ktp manifest-indexer-8b7d5c9f47-lm3zq
shipment-worker-59d4c8b7f6-w8kdt
  FAIL  depot-api: only 0/1 ready - ticket not yet repaired
  FAIL  shipment-worker: only 0/1 ready - ticket not yet repaired
  FAIL  manifest-indexer: only 0/1 ready - ticket not yet repaired
  PASS  depot-tracker: 2/2 ready
  FAIL  DEP-1041 not fixed: image is 'nginx:1.27-alpine' (expected nginx:1.27-alpine)
  FAIL  DEP-1042: shipment-worker restart count is 6 - still crash-looping?
  FAIL  DEP-1043 not fixed: manifest-indexer is still unscheduled (Pending)
  FAIL  DEP-1044 not fixed: Service selector is app='depot-tracking' (expected depot-tracker)
  FAIL  DEP-1044: EndpointSlice still has no addresses

SUMMARY: 17 passed, 9 failed, 0 skipped

```

Exit code 1.

Note depot-tracker: 2/2 ready passes even while DEP-1044 is unfixed.

That is the lesson of Part 5 restated by the script: the Pods really are

healthy. Only the routing is broken, and only the endpoint check catches it.

Other partial runs

kubeconform not installed

```

SKIP  kubeconform not installed - skipping schema validation of broken/
PASS  server-side dry-run: the API server ACCEPTS all broken/ manifests

```

Not a defect. The server-side dry run is the authoritative check; kubeconform is a faster offline equivalent. Install it with `brew install kubeconform` if you want both.

Run before applying broken/

```

FAIL  deployment/depot-api missing - apply broken/ then manifests/20-fixes.yaml

```

Exit code 1. Complete Step 2.1.

Control plane unhealthy

```

FAIL  control plane readyz is 'unreachable' - investigate before continuing

```

Stop and escalate. Nothing in this lab should ever cause this: Part 6 is read-only by design. If you see it, something outside the lab is wrong with the cluster.

Manual observations the script cannot make for you

The script confirms the end state. It cannot confirm that you worked the method, which is the actual assessed skill.

#	Observation	Where
1	All five broken objects created with zero errors - schema-valid is not working	Step 2.1

#	Observation	Where
2	Four different failure phases localised from one kubectl get pods -o wide	Step 2.3
3	kubectl logs on the image-pull Pod returning "waiting to start", not an application error	Step 3.3
4	Pulled/Created/Started all Normal on the crash-looping Pod - proving Q2 passed	Step 4.1
5	Exit code 127 in lastState.terminated.exitCode	Step 4.3
6	FailedScheduling: 0/1 nodes are available with the per-reason tally	Step 4.7
7	Requested 64 cpu / 512Gi against the node's real allocatable	Step 4.8
8	depot-tracker Pods 1/1 Running while the Service is unusable	Step 5.1
9	ENDPOINTS <unset> versus the baseline's two addresses	Step 5.3
10	kubectl get pods -l app=depot-tracking returning No resources found	Step 5.5
11	readyz?verbose line-by-line, and knowing [-]etcd failed is the critical one	Step 6.3
12	The four static Pod manifests in /etc/kubernetes/manifests/ - read, never edited	Step 6.6
13	The scheduler's own log line for DEP-1043, matching the Event you read earlier	Step 6.7
14	Localising the blind Part 8 injection in under 60 seconds	Step 8.3